

Appunti di Sistemi Distribuiti

Corso di Laurea in Informatica A.A 25/26

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

4	Struttura del Web	1
4.1	Ipertesti	1
4.2	Pagine Web e file HTML	1
4.3	Meccanismi di Identificazione Univoca	2
4.3.1	URL - Uniform Resource Locator	2
4.3.2	URN - Uniform Resource Name	3
4.4	I linguaggi del Web	4
4.4.1	Struttura, Stile e Dati	4
4.4.2	Dati Multimediali e Non Testuali	4
4.4.3	Linguaggi di scripting	4
5	Protocollo HTTP - HyperText Transfer Protocol	5
5.1	Modello Client-Server	5
5.1.1	Client	5
5.1.2	Server	5
5.2	Fase di Connessione	5
5.3	Natura "Stateless"	6
5.3.1	Vantaggi	6
5.3.2	Svantaggi	6
5.4	Gestione delle Connessioni	6
5.4.1	HTTP non persistente	6
5.4.2	HTTP persistente	6
5.5	Struttura e Formato dei Messaggi	7
5.5.1	Formato delle Richieste	7
5.5.2	Formato delle Risposte	8
5.6	Metodi HTTP	9
5.6.1	Le proprietà dei metodi	9
5.6.2	Metodo GET	9
5.6.3	Metodo HEAD	9
5.6.4	Metodo OPTIONS	10
5.6.5	Metodo TRACE	10
5.6.6	Metodo POST	10
5.6.7	Metodo PUT	10
5.6.8	Metodo DELETE	11
5.7	Codici di stato	11
5.7.1	Le 5 classi dei codici di stato	11
5.8	I Cookie e il mantenimento dello stato	12
5.8.1	L'architettura dei Cookie	12
5.9	Web Caching	12

5.9.1	Il ruolo del Server Proxy	12
5.10	GET condizionale	13
5.10.1	L'intestazione "if-modified-since"	13
5.11	Message vs Streaming communication	14
5.11.1	Livello di Applicazione (Comunicazione a Messaggi)	14
5.11.2	Livello di Trasporto (Comunicazione a Stream e Segmenti)	14
5.12	Tipi di comunicazione Message-oriented communication	15
5.12.1	Comunicazione Persistente Asincrona	15
5.12.2	Comunicazione persistente sincrona	15
5.12.3	Comunicazione transiente sincrona basata sulla consegna (Delivery based)	15
5.12.4	Comunicazione transiente sincrona basata sulla risposta (Response based)	15
5.13	Modello a Code di Messaggi (Message Queuing)	16
5.13.1	Operazioni per interagire con la coda	16
5.14	Message Broker e Pattern Publish/Subscribe	17
5.14.1	Funzionalità speciali per i sistemi critici (es. protocollo MQTT)	17
6	Formati di scambio	18
6.1	Perchè usarli?	18
6.2	Come usarli?	18
6.2.1	Marshaling (o Serializzazione)	18
6.2.2	Unmarshaling (o Deserializzazione)	18
7	Linguaggi di Markup	19
7.1	Definizione	19
7.2	Tipi di marcatura	19
8	HTML - HyperText Markup Language	20
8.1	Definizione	20
8.2	Cosa fa?	20
8.3	Uso dei Tag e la semantica	21
8.4	Interazione con l'utente: Forum	22
8.5	DOM - Document Object Model	23
8.5.1	Struttura ad Albero	23
8.5.2	Navigazione dei nodi	23
8.5.3	Identificazione dei Nodi	23
8.5.4	Funzioni di ricerca per i nodi	24
8.5.5	Eventi e Listener	24
9	CSS - Cascade Style Sheet	25
9.1	Definizione	25
9.2	Modalità di inclusione	25
9.3	Struttura Base	26
9.3.1	Selettori	26
9.3.2	Combinatori	26
9.3.3	Pseudo-classi	27
9.4	Algoritmo "a Cascata"	27
9.5	Media Query	27
10	XML - eXtensible Markup Language	28

Capitolo 4

Struttura del Web

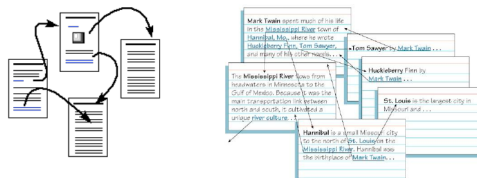
4.1 Iper testi

Il fondamento logico per l'organizzazione delle informazioni sul Web è l'ipertesto (hypertext).

Un ipertesto è un insieme di contenuti (testuali e non) che si caratterizza per la sua non linearità.

Le informazioni, infatti, non vengono lette in modo sequenziale, ma vi si accede seguendo percorsi liberi definiti da collegamenti ipertestuali, i cosiddetti link.

A livello architetturale l'insieme degli ipertesti è rappresentabile come una rete con una struttura a grafo orientato.



4.2 Pagine Web e file HTML

L'ipertesto prende forma concreta nella pagina Web, la quale è composta da diverse risorse (oggetti) di cui il browser deve fare il rendering, come testo, immagini o musica.

La struttura portante e i contenuti della pagina, inclusi i riferimenti alle altre risorse, sono definiti all'interno di un file scritto in HTML (HyperText Markup Language), il linguaggio standard per la scrittura degli ipertesti.

Nel sistema, anche lo stesso file HTML è considerato a tutti gli effetti una risorsa

4.3 Meccanismi di Identificazione Univoca

4.3.1 URL - Uniform Resource Locator

Ogni singola risorsa presente sul Web deve essere identificata in modo univoco.

Lo strumento principale per farlo è 'URL, il cui compito non è solo **identificare**, ma anche localizzare fisicamente la risorsa, indicando in modo preciso dove andarla a trovare.

Struttura dell'URL

Un indirizzo URL si struttura unendo cinque componenti principali :

1. **Nome del protocollo**

Indica quale protocollo usare per il trasferimento dei dati

2. **Indirizzo IP**

L'indirizzo dell'host (o l'host name che andrà risolto tramite DNS)

3. **Porta di destinazione**

Parametro opzionale

4. **Percorso (cammino)**

Il percorso nell'host seguendo una gerarchia a cartelle

5. **Identificatore della risorsa**

Il nome del file finale

protocollo://indirizzo_IP/hostname[:porta]/cammino/risorsa
1. 2. 3. 4. 5.

Figura 4.1: Struttura di un URL

4.3.2 URN - Uniform Resource Name

Mentre l'URL indica dove si trova un oggetto, l'URN è un identificatore che definisce cosa è una risorsa, in modo del tutto indipendente dalla sua posizione fisica nella rete.

Questo garantisce due proprietà essenziali :

- **Persistenza**

Il nome assegnato tramite URN rimane valido anche se la risorsa viene fisicamente spostata su un altro server

- **Astrazione**

L'identificatore è slegato e non contiene alcuna informazione relativa al protocollo di accesso

Struttura dell'URN

La struttura di un URN segue il formato

1. **urn**

Identifica il tipo di identificatore (sempre fisso)

2. **<NID> - Namespace Identifier**

Identifica il tipo di risorsa

3. **<NSS> - Namespace Specific String**

Codice identificativo univoco della risorsa

```
urn:isbn:12345
urn:uuid:f81d4fae-7dec-11d0-a765-00a0c91e6bf6
```

Figura 4.2: esempi di assegnazione identificativi univoci

```
http://biblioteca.it/libri/urn:isbn:9788804668237
http://miosito.it/user/[urn:uuid:]f47ac10b-58cc-4372-a567-
0e02b2c3d479
```

Figura 4.3: esempio di identificazione di una risorsa in URL

4.4 I linguaggi del Web

All'interno dell'architettura del Web, i dati e i documenti testuali vengono espressi utilizzando una serie di linguaggi standardizzati, ognuno con uno scopo ben preciso.

4.4.1 Struttura, Stile e Dati

HTML - HyperText Markup Language

Il linguaggio fondamentale è l'**HTML**, il quale viene impiegato per definire la struttura portante e i contenuti effettivi della pagina Web.

CSS - Cascade Style Sheets

Per gestire l'**aspetto estetico** (rendering visivo, layout e formazione), all'interno delle pagine HTML può essere integrato del codice CSS.

XML e JSON

Quando invece l'obiettivo primario non è la visualizzazione ma la mera rappresentazione dei dati e il loro scambio tra applicazioni, si utilizzano linguaggi standard specifici.

I più importanti e diffusi in questo ambito sono l'**XML** e il **JSON**.

4.4.2 Dati Multimediali e Non Testuali

Oltre al testo, il Web è ricco di contenuti non testuali, come immagini, file audio e documenti video.

Poiché i protocolli come l'HTTP sono orientati al testo, si sfrutta la **codifica MIME**.

Questo meccanismo permette di trasformare file complessi e binari in una stringa di caratteri testuali convenzionali, rendendoli adatti a viaggiare sulla rete.

4.4.3 Linguaggi di scripting

Per fare in modo che una pagina Web non sia statica ma possa offrire funzionalità avanzate e arricchire l'interazione con l'utente, essa può contenere al suo interno del codice eseguibile.

Questo codice è tipicamente espresso in linguaggi di scripting, principalmente **Javascript**.

Capitolo 5

Protocollo HTTP - HyperText Transfer Protocol

L'HTTP è il protocollo di livello applicativo fondamentale per il Web.

5.1 Modello Client-Server

In questo ecosistema, le responsabilità sono divise in :

5.1.1 Client

È tipicamente rappresentato da un browser web.

Il suo compito è inviare richieste e occuparsi del rendering contenuti per l'utente, un processo che può adattarsi a dispositivi diversi, inclusi quelli "non visuali".

5.1.2 Server

È il Web server, che riceve le richieste e invia gli oggetti corrispondenti in risposta.

Un aspetto importante è che oggetti diversi che compongono una singola pagina Web possono essere prelevati anche da server Web differenti.

5.2 Fase di Connessione

L'HTTP non trasporta fisicamente i dati, ma si affida al protocollo di trasporto TCP.

La comunicazione avviene attraverso questa procedura :

1. Il client avvia una connessione TCP(creando una socket) verso l'indirizzo del server, tipicamente sulla porta 80.
2. Il server accetta la connessione.
3. Client e Server si scambiano i messaggi HTTP.
4. La connessione TCP viene chiusa.

5.3 Natura "Stateless"

Una delle scelte architetturali più rilevanti dell'HTTP è l'essere un protocollo **stateless**, ovvero privo di stato

5.3.1 Vantaggi

Il server non mantiene in memoria alcuna informazione sulle richieste precedenti del client.

Questo alleggerisce i server, evitando la complessità dei protocolli che mantengono lo stato.

5.3.2 Svantaggi

Ogni singola richiesta deve contenere tutte le informazioni necessarie per la sua esecuzione, aumentando l'overhead.

Per ovviare a questo limite e gestire sessioni HTTP si avvale dell'uso dei Cookie

5.4 Gestione delle Connessioni

La gestione delle connessioni TCP fa una grande differenza sulle prestazioni e si divide in 2 approcci principali

5.4.1 HTTP non persistente

Storicamente utilizzato in HTTP/1.0.

Questo modello prevede che su una connessione TCP appena aperta venga inviato al massimo un solo oggetto, dopodiché la connessione viene chiusa.

Se una pagina necessita di 10 immagini, il client dovrà aprire e chiudere connessioni ripetutamente.

Il tempo di risposta per scaricare ogni singolo oggetto è pari a **2 RTT (Round Trip Time) + il tempo di trasmissione dell'oggetto**

5.4.2 HTTP persistente

Adottato da HTTP/1.1, HTTP/2.0 e HTTP/3.0 questo modello lascia la connessione aperta.

Sfruttano un'unica connessione, il server può inviare al client più oggetti in sequenza.

In condizione in cui l'RTT è molto maggiore del tempo di trasmissione, questo approccio dimezza all'incirca i tempi di caricamento per ogni risorsa successiva alla prima.

5.5 Struttura e Formato dei Messaggi

HTTP è un protocollo orientato al carattere, di conseguenza, il formato dei suoi messaggi si basa su testo leggibile (ASCII).

5.5.1 Formato delle Richieste

Esempio messaggio di richiesta

```
GET /index.html HTTP/1.1\Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0)
Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
```

Caratteri speciali

Ogni messaggio di richiesta è strutturato in una sequenza precisa di righe separate tra loro tramite due speciali caratteri di controllo

- **cr-carriage return (ritorno a capo)**
- **lf-Line feed (nuova linea)**

Riga di richiesta

È la riga di apertura del messaggio.

Contiene 3 parametri separati da uno spazio (sp)

- Il **metodo** HTTP che il client vuole eseguire
- L'**URL** identificativo della risorsa
- La **versione** del protocollo HTTP in uso

Esempio: Riga di richiesta

```
GET /index.html HTTP/1.1\r\n
```

Righe di intestazione

Subito dopo la riga di richiesta, si collocano gli header.

Sono strutturati nel formato **nome-campo: valore** e seguiti da un **cr lf**.

Servono al client per passare vari parametri al server, come ad esempio :

- **Host**
L'host specifico in cui risiede l'oggetto
- **User-Agent**
Il tipo di browser in uso

- **Accept**

Preferenze sulla negoziazione del contenuto tramite formati MIME

- **Preferenze aggiuntive** Come la lingua (Accept-Language) o il tipo di compressione accettata (Accept-Encoding)

Esempio Righe di intestazione

```
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0)
Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
```

5.5.2 Formato delle Risposte

I messaggi restituiti dal server iniziano con una riga di stato, seguita dalle righe di intestazione e infine dai dati effettivi (come il file HTML o l'immagine richiesta)

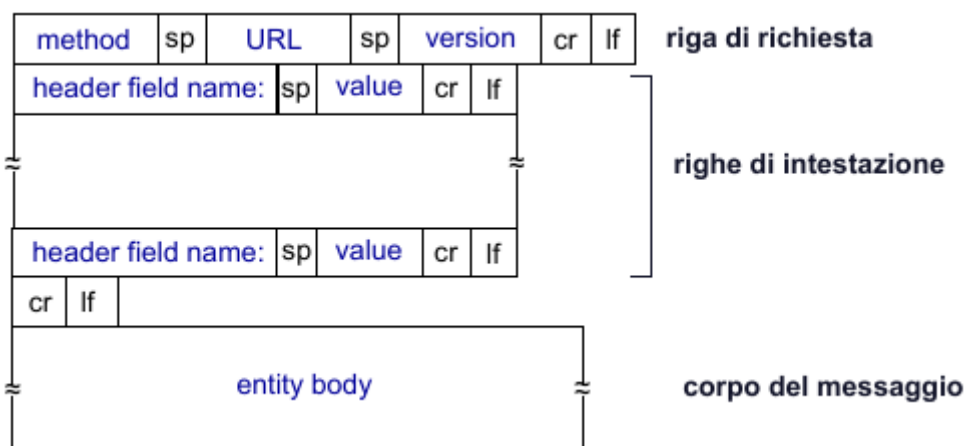


Figura 5.1: Struttura del messaggio generale

5.6 Metodi HTTP

I metodi HTTP definiscono l'azione specifica che il client desidera effettuare sulla risorsa indicata dall'URL.

Prima di elencarli è essenziale comprendere 2 parametri fondamentali con cui questi metodi vengono classificati

5.6.1 Le proprietà dei metodi

I metodi si distinguono in base a 2 proprietà che descrivono il loro impatto sui dati presenti nel server

Metodi Sicuri

Un metodo viene definito sicuro se la sua esecuzione non modifica in alcun modo lo stato del server.

Serve, in sostanza solo per la lettura

Metodi Idempotenti

Un metodo è considerato idempotente quando l'effetto che ha sul server dopo una singola richiesta andata a buon fine è esattamente identico all'effetto che si otterrebbe eseguendo più richieste identiche in modo consecutivo.

5.6.2 Metodo GET

Il metodo GET richiede al server di recuperare la risorsa specifica dall'URL o qualunque informazione a essa relativa.

È il metodo più comune per ottenere dati in vari formati, come documenti HTML, XML, JSON o contenuti multimediali.

Se la richiesta necessita di parametri, questi vengono agganciati direttamente in coda all'URL.

Proprietà

Trattandosi di un metodo di sola lettura, la GET è classificata come metodo **sicuro** e **idempotente**.

5.6.3 Metodo HEAD

La HEAD è praticamente identica alla GET, ma con una differenza cruciale : il server non deve restituire il corpo del messaggio nella risposta, ma solo le righe di intestazione.

L'uso tipico di questo metodo è il **debugging** oppure l'ottenimento di informazioni meta-dati su una risorsa

Proprietà

Esattamente come la GET, anche la HEAD è un metodo **sicuro** e **idempotente**.

Esempio di GET

```
GET myCompany/orders?item="Q2345"&date="2022/03" HTTP1.1
```

5.6.4 Metodo OPTIONS

Questo metodo rappresenta una richiesta per ottenere informazioni sulle opzioni di comunicazione permesse relativamente alla risorsa identificata dall'URL.

Proprietà

È un metodo **sicuro** e **idempotente**.

5.6.5 Metodo TRACE

Viene utilizzato per invocare un loopback del messaggio di richiesta a livello applicativo da parte del server, utile anch'esso per scopi diagnostici.

Proprietà

È un metodo **sicuro** e **idempotente**.

5.6.6 Metodo POST

La POST viene utilizzata per comunicare al server dei dati che devono essere elaborati, o per richiedere che il server accetti la risorsa inclusa nella richiesta come un nuovo "subordinato dall'URL indicato.

A differenza della GET, i dati non vengono passati nell'URL, ma sono inseriti come documento autonomo all'interno del corpo della richiesta, subito dopo le righe di intestazione.

L'uso tipico è trasportare i dati compilati da un utente all'interno di un form.

Proprietà

Poichè la POST serve tipicamente a creare o aggiornare dati sul server, **non è un metodo sicuro e non è idempotente**.

Esempio di POST

```
POST myCompany/orders HTTP1.1
Content-Length: 59
Content-Type: application/x-www-form-urlencoded
update=true&oldItem="Q2345"&newItem="Q68254"&date="2022/04"
```

5.6.7 Metodo PUT

La PUT richiede che la risorsa fornita venga memorizzata e caricata sul server all'URL specificato.

La sua particolarità è che, se nell'URL indicato esiste già un file, questo **viene completamente sostituito** dal nuovo contenuto presente nel corpo della richiesta.

Proprietà

Modificando i file, la PUT non è un metodo sicuro.

Tuttavia a differenza della POST, **è un metodo idempotente** : caricare o sovrascrivere lo stesso file una volta oppure cento volte porta sempre all'identico stato finale sul server

5.6.8 Metodo DELETE

Come deducibile dal nome, questo metodo richiede esplicitamente al server di **eliminare la risorsa** identificata dall'URL.

Proprietà

Non essendo un'operazione di sola lettura, non è un metodo sicuro.

Tuttavia, **è un metodo idempotente**: richiedere l'eliminazione di un file già eliminato non comporta ulteriori cambiamenti di stato nel server.

5.7 Codici di stato

I codici di stato sono valori numerici, accompagnati da una breve descrizione testuale, restituiti dal Web server al client all'interno dei messaggi di risposta HTTP.

La loro funzione è comunicare l'esito dell'operazione richiesta.

Fisicamente, si trovano nella riga di stato, ovvero la primissima riga del messaggio di risposta inviato dal server

5.7.1 Le 5 classi dei codici di stato

I codici sono suddivisi rigorosamente in 5 diverse classi, categorizzate in base al 1° numero della cifra.

1xx (Informational)

I codici che iniziano con 1 indicano una comunicazione di tipo informativo.

Segnalano al client che la richiesta è stata ricevuta e che il server sta continuando regolarmente la sua elaborazione.

2xx (Success)

I codici che iniziano con 2 vengono utilizzati per confermare un successo.

Indica che la richiesta del client è stata correttamente ricevuta, compresa, accettata ed elaborata dal server.

3xx (Redirection)

Questa classe di codici segnala che la risorsa non può essere fornita direttamente, ma che è necessario che il client intraprenda ulteriori azioni per poter completare la richiesta in modo corretto.

4xx (Client Error)

I codici 4xx indicano che il problema è imputabile a chi ha fatto la richiesta.

Segnalano che la richiesta inviata dal client presenta una sintassi errata oppure risulta incomprensibile al server.

5xx (Server Error)

I codici 5xx indicano un errore imputabile all'infrastruttura di destinazione.

Segnalano che il server non è riuscito a soddisfare una richiesta che risultava apparentemente valida.

5.8 I Cookie e il mantenimento dello stato

Per risolvere il limite imposto dall'assenza di stato, i client e i server HTTP si avvalgono dei cookie per riuscire a mantenere in memoria lo stato nelle transizioni.

Questa tecnologia viene impiegata tipicamente per 3 scopi principali:

1. Gestione dell'autorizzazione e autenticazione
2. Mantenimento del carrello negli acquisti online
3. Creazione di raccomandazioni (come ad esempio le pubblicità mirate)

5.8.1 L'architettura dei Cookie

L'intero meccanismo di funzionamento dei cookie si basa su 4 componenti fondamentali che lavorano in sinergia:

1. Una **riga di intestazione specifica presente nella risposta HTTP** generata dal server
2. Una **riga di intestazione specifica inserita nella successiva richiesta HTTP** da parte del client.
3. Un **file locale relativo al cookie**, conservato sul dispositivo del client e gestito dal suo browser.
4. Un **database di backend** mantenuto e gestito attivamente dal Web server

5.9 Web Caching

L'obiettivo principale del Web caching è soddisfare le richieste dei client utilizzando delle copie salvate localmente, invece di dover scaricare i file dal server Web di origine.

Queste copie possono risiedere in 2 posti:

1. Nella **Web cache locale del dispositivo**, gestita direttamente dal browser dell'utente
2. In appositi server disseminati nelle reti istituzionali o di accesso, noti come **Web cache** o **server proxy**

5.9.1 Il ruolo del Server Proxy

Quando un proxy è attivato, il browser invia tutte le sue richieste HTTP direttamente a questo nodo intermedio.

Il server proxy svolge un doppio ruolo:

- **Si comporta da Server**

Se possiede già in memoria l'oggetto richiesto dal client, lo invia immediatamente

- **Si comporta da Client**

Se l'oggetto non è presente nella sua cache, lo richiede al server Web di origine, lo salva e infine lo inoltra al client che lo aveva richiesto

È il server di origine a dettare le regole alla cache: attraverso intestazioni specifiche nella risposta HTTP, comunica se un file può essere salvato e per quanto tempo

5.10 GET condizionale

Il problema principale delle cache è capire se il file salvato in memoria è ancora la versione più recente, oppure se il server di origine lo ha aggiornato.

Per risolvere questo problema, il protocollo HTTP utilizza una richiesta speciale detta "GET condizionale".

5.10.1 L'intestazione "if-modified-since"

Quando il client (o il proxy) richiede un file che ha già in cache ma di cui vuole verificare la validità, inserisce nella richiesta HTTP un parametro specifico: `if-modified-since: <data>`

Con questa riga, il client sta dicendo al server:

"Inviarmi questo file solo se è stato modificato dopo questa precisa data (ovvero la data in cui ho scaricato la mia copia locale)"

Possibili risposte del Server

A fronte di una GET condizionale, il server Web controlla il file originale e può rispondere in due modi:

- **Se il file NON è stato modificato**

Il server genera un messaggio di risposta con il codice di stato 304 Not Modified.

La particolarità fondamentale è che **questa risposta è vuota**, ovvero non include alcun oggetto al suo interno.

Questo permette di risparmiare un'enorme quantità di banda e tempo di trasferimento, autorizzando il client a usare la copia che ha già in memoria.

- **Se il file È stato modificato**

Il server elabora la richiesta come una normale GET.

Risponde con un codice 200 OK e inserisce nel corpo del messaggio la nuova versione aggiornata dei dati.

5.11 Message vs Streaming communication

La differenza principale tra la comunicazione a messaggi e quella a stream risiede nel modo in cui i dati vengono elaborati e percepiti dai diversi livelli dell'architettura di rete

5.11.1 Livello di Applicazione (Comunicazione a Messaggi)

A livello applicativo, come nel caso del protocollo HTTP, la comunicazione è **orientata al messaggio**.

Questo significa che i software comunicano scambiandosi pacchetti di dati interi e ben definiti (i messaggi).

5.11.2 Livello di Trasporto (Comunicazione a Stream e Segmenti)

Per il livello di trasporto, i messaggi non esistono come entità logiche, ma sono visti nel loro complesso come un **flusso di byte da trasportare in rete**.

Il livello di trasporto prende questo stream e lo scompone in **segmenti**.

La gestione di questi segmenti dipende dal protocollo utilizzato:

- **Servizio UDP**

Il protocollo scompone lo stream di byte in segmenti e li inoltra al livello di rete in **modalità best-effort**

- **Servizio TCP**

Anche il TCP scompone e invia lo stream come segmenti, ma **ogni segmento viene numerato**.

Questa numerazione permette a TCP di garantire 3 funzionalità cruciali:

1. **Riordinamento**

I segmenti arrivati fuori sequenza vengono rimessi in ordine giusto.

2. **Controllo delle duplicazioni**

I segmenti ricevuti più volte vengono scartati

3. **Controllo delle perdite**

Il sistema si accorge dei buchi nella numerazione e richiede il rinvio dei segmenti mancanti

5.12 Tipi di comunicazione Message-oriented communication

La differenza fondamentale tra i vari tipi di comunicazione si basa sul **punto di sincronizzazione**, ovvero il momento esatto in cui il client smette di attendere una conferma e riprende regolarmente la propria esecuzione.

5.12.1 Comunicazione Persistente Asincrona

- **Punto di sincronizzazione**

Avviene all'**invio della richiesta**

- **Funzionamento**

Il mittente sospende la sua esecuzione solo per il tempo necessario affinché il messaggio venga preso in carico dal proprio sistema intermedio locale (il middleware lato mittente), e non attende alcuna risposta dal destinatario finale

- **Esempio**

Il classico sistema di posta elettronica, in cui il client attende solo che il proprio server SMTP accetti l'email da spedire.

5.12.2 Comunicazione persistente sincrona

- **Punto di sincronizzazione**

Avviene all'inoltro della richiesta

- **Funzionamento**

Il mittente invia il messaggio e attende.

Riprenderà la sua esecuzione solo quando il software middleware situato fisicamente sul lato del ricevente avrà ricevuto il messaggio e avrà inviato una notifica di avvenuta consegna.

5.12.3 Comunicazione transiente sincrona basata sulla consegna (Delivery based)

- **Punto di sincronizzazione**

Avviene all'inizio dell'**elaborazione della richiesta**(at request processing)

- **Funzionamento**

Il mittente rimane bloccato in attesa finché l'applicazione destinataria (che deve essere attiva) gli notifica di aver effettivamente iniziato a elaborare la richiesta ricevuta.

- **Esempio**

L'invio di comandi a un server remoto tramite protocollo SSH

5.12.4 Comunicazione transiente sincrona basata sulla risposta (Response based)

- **Punto di sincronizzazione**

Avviene dopo l'elaborazione della richiesta (after processing)

- **Funzionamento**

È il modello che richiede l'attesa maggiore.

Il mittente rimane in sospeso e continua l'esecuzione solo quando il destinatario ha elaborato con successo e in modo completo la richiesta, restituendo la risposta finale.

- **Esempio**

Il protocollo HTTP in cui il browser attende di ricevere dal server la risorsa richiesta.

5.13 Modello a Code di Messaggi (Message Queuing)

In questo modello, le applicazioni non comunicano passandosi i dati direttamente, ma inseriscono i messaggi all'interno di code specifiche gestite da appositi gestori (queue manager)

Questa architettura si basa su 2 caratteristiche fondamentali:

- **Persistenza (Storage)**

Il sistema ha la capacità di memorizzare fisicamente i messaggi.

Una volta che un messaggio viene inserito all'interno di una coda, vi rimane al sicuro finché non viene esplicitamente prelevato e rimosso.

- **Disaccoppiamento temporale (Asincronismo)**

Non è assolutamente necessario che il mittente e il destinatario siano in esecuzione nello stesso momento.

Il mittente può generare un messaggio che se il ricevente è spento e, viceversa, il ricevente può recuperarlo dalla sua coda molto tempo dopo che il mittente ha finito di lavorare

5.13.1 Operazioni per interagire con la coda

Per interagire con queste code, l'interfaccia di comunicazione fornisce **4 primitive di base**.

1. **PUT**

Inserisce un messaggio nella coda di invio (operazione non bloccante).

2. **GET**

Rimuove il primo messaggio disponibile dalla coda di ricezione (operazione bloccante, il ricevente si ferma ad aspettare finché non arriva un messaggio).

3. **POLL**

Controlla in modo non bloccante la presenza di messaggi e, se ci sono, rimuove il primo.

4. **NOTIFY**

Installa un "handler" (una funzione di callback) che viene richiamato automaticamente dal middleware non appena un nuovo messaggio inserito nella coda avvisando l'applicazione.

5.14 Message Broker e Pattern Publish/Subscribe

Il pattern Publish/Subscribe è un'evoluzione che utilizza un nodo centrale chiamato **Message Broker** per gestire lo smistamento delle informazioni, basandosi sugli **argomenti** (Topic) anziché sugli indirizzi di destinazione.

L'architettura funziona in questo modo:

- **I Publisher (pubblicatori)**

Non sanno chi riceverà i loro dati, si limitano a collegarsi al broker dichiarando:

"voglio inviare messaggi relativi a questo specifico Topic".

- **I Subscriber (sottoscrittori)**

A loro volta, si registrano al broker dichiarando:

"voglio ricevere tutti i messaggi pubblicati su questo Topic".

Questa meccanica garantisce una **totale indipendenza** tra chi produce i dati e chi li consuma, rendendo facilissimo creare reti di comunicazione complesse del tipo uno-a-molti, molti-a-uno o molti-a-molti.

Se per un certo topic non ci sono subscriber attivi in quel momento, il broker può mantenere i dati in memoria applicando diverse politiche.

5.14.1 Funzionalità speciali per i sistemi critici (es. protocollo MQTT)

Poiché il modello pub/sub è massicciamente utilizzato nell'Internet of things e in ambienti critici, i broker offrono funzionalità avanzate:

- **Messaggio "Retain"**

Il publisher può chiedere al broker di conservare uno e un solo messaggio per un topic.

Grazie al flag retain, non appena un nuovo subscriber si connette, riceve immediatamente quest'ultimo valore valido senza dover aspettare una nuova pubblicazione.

- **Last Will and Testament (LWT)**

Al momento della connessione, il publisher può consegnare al broker un "testamento".

Se il dispositivo publisher dovesse subire una disconnessione inaspettata, il broker inoltrerà automaticamente questo messaggio di allarme a tutti i subscriber.

È una funzione fondamentale per accorgersi tempestivamente di un'anomalia nei sistemi critici.

Capitolo 6

Formati di scambio

6.1 Perché usarli?

Nei sistemi distribuiti, le diverse componenti sono spesso sviluppate in linguaggi di programmazione differenti e vengono eseguite su sistemi operativi o piattaforme eterogenee.

Poiché questi sistemi non dividono la stessa memoria fisica, i dati devono necessariamente viaggiare sulla rete sotto forma di un flusso ininterrotto di byte.



Figura 6.1: Procedura di comunicazione tra client e server attraverso formati di scambio

6.2 Come usarli?

Per far sì che sistemi che non si conoscono possano comunicare e capirsi, è fondamentale concordare un "contratto" comune, ovvero un formato di scambio dei dati che sia totalmente indipendente dalla piattaforma utilizzata, come ad esempio i formati di testo XML o JSON.

Per trasferire attraverso la rete delle strutture dati complesse in modo sicuro, le applicazioni si avvolgono di due operazioni speculari:

6.2.1 Marshaling (o Serializzazione)

È il processo mediante il quale l'applicazione **mittente** converte le proprie strutture dati residenti in memoria in un formato trasmissibile (come testo o byte) per poi inviarlo sulla rete o salvarlo su un file.

6.2.2 Unmarshaling (o Deserializzazione)

È il processo inverso eseguito dal **destinatario**. Ricevuto il flusso di dati serializzato, il sistema ricevente lo "decompacchetta" e ricostruisce in memoria la struttura dati logica originale per poterla elaborare correttamente.

Capitolo 7

Linguaggi di Markup

7.1 Definizione

Sono sistemi di annotazione che usano speciali etichette (tag) per descrivere in modo esplicito la semantica e la struttura dei documenti.

7.2 Tipi di marcatura

Esistono 3 tipologie principali di marcatura:

- **Descrittive**

Etichettano semplicemente parti del testo, disaccoppiando la struttura dalla presentazione del testo stesso.

- **Procedurali**

Definiscono istruzioni per programmi che elaborino il testo al quale sono associate.

- **Presentazione**

Definiscono come visualizzare il testo al quale sono associate.

Capitolo 8

HTML - HyperText Markup Language

8.1 Definizione

L'HTML è un linguaggio di marcatura basato su etichette che servono ad annotare un documento in modo che queste annotazioni siano sintatticamente distinguibili dal testo vero e proprio.

8.2 Cosa fa?

Lo scopo principale dell'HTML è quello di fornire la struttura logica e descrivere la semantica dei contenuti presenti in una pagina Web.

Il suo ruolo all'interno dell'ecosistema Web è strettamente legato al concetto di **separazione delle responsabilità**, l'HTML ha il compito esclusivo di strutturare gerarchicamente l'informazione, garantendo la totale separazione tra il contenuto e la sua presentazione, la quale viene delegata a CSS.

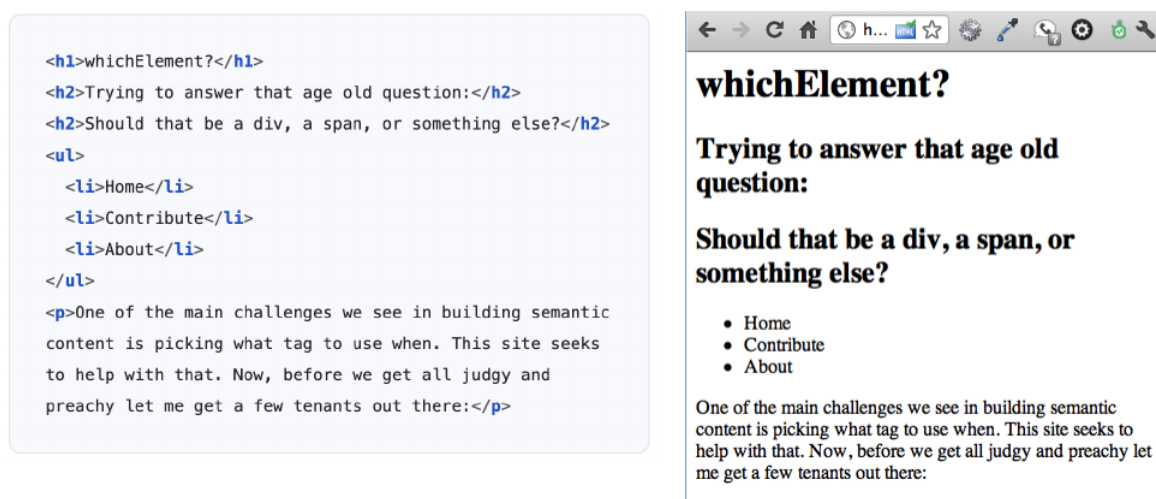


Figura 8.1: Esempio di codice HTML e il suo risultato

8.3 Uso dei Tag e la semantica

L'HTML struttura il documento racchiudendo le parti di testo all'interno di specifiche etichette di apertura e chiusura.

L'uso corretto di questi tag conferisce un significato preciso al testo contenuto

Tag	Significato
<h1>--<h6>	Intestazioni gerarchiche
<p>	Paragrafo
, ,	Liste (non ordinata, ordinata, elemento)
<table>	Dati tabulari
<form>, <input>	Form interattivi
<a>	Collegamento ipertestuale
, 	Enfasi semantica
 e sono semantici (significano <u>enfasi</u> e importanza), a differenza di <i> e che sono solo visivi (corsivo e grassetto senza significato).	

8.4 Interazione con l'utente: Forum

Quando è necessario richiedere dati all'utente, l'HTML utilizza una struttura specifica chiamata forma, definita dal tag `<form>`.

All'interno del form, l'HTML mette a disposizione svariati widget di interfaccia, come caselle di testo, checkbox, bottoni e menu a tendina

Una volta raccolti i dati, il form li invia a uno script lato server (il cui indirizzo è specificato nell'attributo `action`). L'invio (definito dall'attributo `method`) avviene principalmente in 2 modalità:

- **Metodo GET**

Accoda i parametri inseriti dall'utente direttamente alla fine dell'URL della pagina.

- **Metodo POST**

Inserisce i dati in modo invisibile all'interno del corpo della richiesta HTTP.

Esempio codice di un form

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4
5 <h2>HTML Forms</h2>
6
7 <form action="/action_page.php">
8   <label for="fname">First name:</
   label><br>
9   <input type="text" id="fname"
   name="fname" value="John"><br>
10  <label for="lname">Last name:</
   label><br>
11  <input type="text" id="lname"
   name="lname" value="Doe"><br><br>
12  <input type="submit" value="
   Submit">
13 </form>
14
15 <p>If you click the "Submit" button
   ...</p>
16
17 </body>
18 </html>
```

Example

First name:

Last name:

[Try it Yourself »](#)

8.5 DOM - Document Object Model

DOM è un'interfaccia neutrale rispetto al linguaggio di programmazione e alla piattaforma utilizzata, per consentire ai programmi l'accesso e la modifica dinamica di contenuto, struttura e stile di un documento Web.

8.5.1 Struttura ad Albero

Quando il browser riceve la pagina HTML, non la lascia sotto forma di testo, ma la converte in una complessa struttura ad albero di oggetti.

All'interno di questo albero, ogni singolo tag HTML si trasforma in un "nodo", stabilendo così delle **relazioni gerarchiche di parentela**.

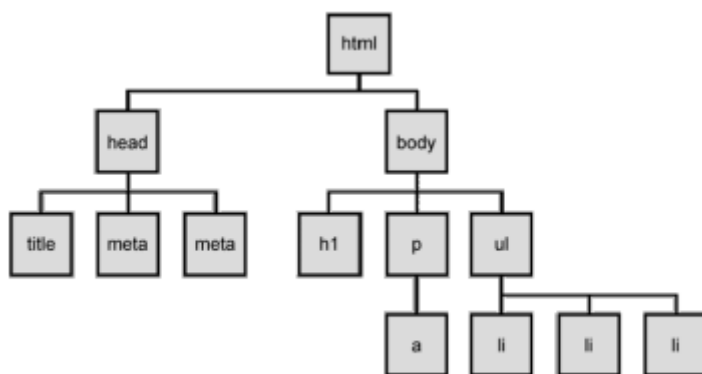


Figura 8.2: Esempio di albero DOM

8.5.2 Navigazione dei nodi

Per permettere ai linguaggi di scripting di muoversi agilmente all'interno di questa architettura, il DOM mette a disposizione diverse proprietà specifiche:

- **parentNode**: Raggiunge il nodo genitore diretto.
- **childNodes**: Restituisce la lista di tutti i nodi figli.
- **firstChild** e **lastChild**: Puntano rispettivamente al primo e all'ultimo figlio del nodo.
- **nextSibling** e **previousSibling**: Permettono di spostarsi orizzontalmente sul nodo fratello successivo o su quello precedente.

8.5.3 Identificazione dei Nodi

Prima di poter manipolare un elemento della pagina, è necessario "trovarlo" all'interno dell'albero.

Ogni nodo può essere caratterizzato da attributi utili a questo scopo:

- Un **identificatore univoco (ID)**, anche se il DOM non ne garantisce l'assoluta unicità a livello tecnico
- Una **classe**, che indica l'appartenenza a un gruppo specifico

8.5.4 Funzioni di ricerca per i nodi

Per sfruttare questi attributi, il browser stesso fornisce nativamente delle funzioni di ricerca rapida, tra cui le più importanti sono:

- **getElementById(idName)**: per estrarre il singolo nodo associato a quel preciso ID.
- **getElementsByClassName(className)**: per estrarre tutti i nodi associati a una specifica classe.

8.5.5 Eventi e Listener

Il DOM non serve solo a modificare passivamente la pagina, ma anche a farla reagire alle azioni dell'utente.

Questo avviene tramite gli **eventi**, ovvero dei segnali emessi dal sistema quando accade qualcosa nel documento.

Javascript può mettersi in ascolto di questi eventi e reagire di conseguenza agganciato delle funzioni specifiche chiamate **listener**.

Categoria	Eventi principali
Mouse	click, dblclick, mouseover, mouseout
Tastiera	keydown, keyup, keypress
Form	submit, change, focus, blur, input
Documento	DOMContentLoaded, load, resize, scroll
Touch	touchstart, touchend, touchmove

```
1 // Seleziono un elemento
2 const btn = document.getElementById("myBtn");
3
4 // Aggiungo un listener
5 btn.addEventListener("click", function(e){
6     console.log("cliccato", e.target);
7 })
```

Esempio di codice

Capitolo 9

CSS - Cascade Style Sheet

9.1 Definizione

Il CSS è il linguaggio utilizzato per definire lo stile e la presentazione visiva dei contenuti web.

Il suo obiettivo primario è concretizzare il principio di "separation of concerns" garantendo una netta divisione tra il **contenuto** e la **vesta estetica**

9.2 Modalità di inclusione

Il codice CSS può essere applicato a un documento HTML i 3 modi differenti:

- **Inline**

Inserito direttamente sul singolo tag HTML tramite l'attributo `style=""`.

- **Interno**

Definito all'interno di un blocco di tag `<style>` situato nella sezione `<head>` della pagina.

- **Esterno**

Scritto all'interno di un file `.css` separato e collegato alla pagina HTML tramite il tag `<link rel="stylesheet" href="theme.css">`.

Questo è il modello preferito perchè separa fisicamente contenuto e stile.

9.3 Struttura Base

Una regola CSS è composta da un "selettore" seguito da un blocco di dichiarazione racchiuse tra parentesi graffe.

Ogni dichiarazione è una coppia proprietà/valore (selettoreproprietà: valore;)

Esempio codice CSS

```
1 h1{
2   color: red;
3   font-size: 2em;
4 }
```

9.3.1 Selettori

I selettori sono il meccanismo con cui il CSS individua precisi elementi o gruppi di elementi nell'albero DOM a cui applicare il design.

Selettori fondamentali

- **Elemento (Tag)**: Seleziona tutti gli elementi di un certo tipo.
- **Classe**: Individua gli elementi che possiedono uno specifico attributo class, preceduto da un punto.
- **ID**: Punta a un elemento univoco contrassegnato da uno specifico attributo id, preceduto dal cancelletto.
- **Universale**: L'asterisco (* . . .) permette di selezionare indiscriminatamente tutti gli elementi della pagina

9.3.2 Combinatori

I combinatori servono per selezionare elementi non per il tag che sono, ma per la loro posizione all'interno dell'albero DOM

- **Discendente**: div p seleziona tutti i paragrafi contenuti dentro un div.
- **Figlio diretto**: ul > li seleziona esclusivamente gli elementi della lista che sono figli diretti del tag ul
- **Gruppo**: h1, h2, h3 consente di raggruppare selettori diversi applicando la stessa regola in un colpo solo

9.3.3 Pseudo-classi

Un elemento HTML è sempre lo stesso tag, ma a livello di interfaccia può cambiare stato.

Le pseudo classi permettono di stilizzare questi stati dinamicamente, senza alterare l'HTML, esempi notevoli sono:

- **a: hover**
- **input: focus**
- **li: first-child**

9.4 Algoritmo "a Cascata"

Poichè esistono diversi fogli di stile, è del tutto normale che le regole si sovrappongano e generino dei conflitti.

Il termine "Cascading" si riferisce all'algoritmo che decide quale regola abbia la precedenza.

La priorità viene valutata secondo quest'ordine:

1. **Importanza**

L'eventuale utilizzo del flag esplicito `!important` associato a un attributo.

2. **Specificità**

Dipende dal selettore utilizzato.

Un ID è più specifico di una classe, che a sua volta prevale sul selettore di un semplice elemento.

3. **Ordine nel sorgente**

A parità di specificità, vince la regola dichiarata per ultima nel file.

9.5 Media Query

Per far sì che la grafica si adatti ai diversi dispositivi, si utilizzano le "media query".

Questi costrutti CSS applicano stili in modo condizionale verificando le caratteristiche del dispositivo in uso, come la larghezza, l'altezza, la risoluzione e l'orientamento dello schermo.

Ad esempio, `@media screen and (min-width: 480px)`

Capitolo 10

XML - eXtensible Markup Language